المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES - TAROUDANT

RAPPORT DE STAGE D'INITIATION

Filière : « Ingénierie Logicielle »

SMARTTRANS-AI

Plateforme de gestion de transport enrichie par l'intelligence artificielle

Présenté par

Hamza Bendaib

Soutenue le _____ à _____

Devant le jury

Pr	_____, professeur à l'ENSIASD de Taroudant	Président
Pr	_____, professeur à l'ENSIASD de Taroudant	Examinateur
Pr	_____, professeur à l'ENSIASD de Taroudant	Examinateur
Mme	Loubna Karbil, professeure à l'ENSIASD de Taroudant	Encadrante pédagogique
Mme	Sara Zaqi, établissement d'accueil : S. WATRANS	Encadrante

Année universitaire 2025–2026

DÉDICACES

Je dédie ce travail à ma famille, pour son amour, son soutien constant et ses encouragements qui m'ont accompagné tout au long de mon parcours académique. Votre présence et votre confiance ont été une source de motivation et de persévérance dans les moments les plus difficiles.

À mes enseignants de l'ENSIASD de Taroudant, pour leur engagement, leur patience et les connaissances qu'ils m'ont transmises, notamment dans les domaines de l'informatique, de l'ingénierie logicielle et de l'intelligence artificielle. Leur enseignement a contribué à construire les bases qui m'ont permis de mener à bien ce projet.

À mes encadrants de stage, pour leur confiance, leurs conseils et leur accompagnement durant la conception et la réalisation de SmartTrans-AI. Leurs orientations et leurs remarques m'ont permis d'enrichir mes connaissances et de développer davantage mes compétences professionnelles.

Enfin, je dédie ce travail à toutes les personnes qui, de près ou de loin, m'ont encouragé et soutenu tout au long de cette expérience.

REMERCIEMENTS

Je tiens à remercier en premier lieu mon encadrante pédagogique, **Mme Loubna Karbil**, pour son suivi tout au long de ce stage, ses conseils, sa disponibilité et ses orientations méthodologiques. Son accompagnement m'a permis de mieux structurer mon travail, d'approfondir ma réflexion et de mener à bien les différentes étapes de ce projet.

Mes sincères remerciements s'adressent également à mon encadrante au sein de **S. WATRANS**, **Mme Sara Zaqi**, pour m'avoir accueilli au sein de l'entreprise et plus particulièrement au sein du département informatique. Je la remercie pour la confiance qu'elle m'a accordée en me confiant le projet **SmartTrans-AI**, ainsi que pour sa disponibilité, ses conseils et les échanges qui m'ont permis de mieux comprendre les besoins et les problématiques liés au domaine du transport et de la gestion de flotte.

Je souhaite également exprimer ma gratitude à l'ensemble des collaborateurs de **S. WATRANS** pour leur accueil, leur disponibilité et leur bienveillance durant toute la période de mon stage. Leur collaboration et leurs explications m'ont permis de découvrir davantage le fonctionnement d'une entreprise et de mieux comprendre l'importance des outils informatiques dans la gestion quotidienne de ses activités.

Ce stage a constitué pour moi une expérience particulièrement enrichissante, aussi bien sur le plan technique que professionnel. Il m'a permis de mettre en pratique les connaissances acquises durant ma formation, notamment dans les domaines du développement logiciel, des bases de données, de la conception des systèmes et de l'intelligence artificielle. Il m'a également permis de développer mon autonomie, mon esprit d'analyse, ma capacité à résoudre des problèmes et mon aptitude à travailler sur un projet répondant à des besoins réels.

Je remercie également l'ensemble du corps professoral de l'**ENSIASD de Taroudant** pour la qualité de l'enseignement dispensé et pour les connaissances et compétences acquises au cours de ma formation. Ces acquis ont constitué une base importante pour la réalisation de ce stage et du projet qui m'a été confié.

Je tiens aussi à remercier mes camarades de promotion pour leur soutien, leurs échanges d'idées et les discussions techniques qui ont contribué à rendre cette expérience plus enrichissante.

Enfin, j'adresse mes plus sincères remerciements à ma famille pour son soutien constant, ses encouragements, sa patience et sa confiance. Leur accompagnement m'a donné la motivation nécessaire pour avancer et donner le meilleur de moi-même tout au long de cette période.

À toutes les personnes qui ont contribué, directement ou indirectement, à la réussite de ce stage et à la réalisation de ce projet, je leur adresse mes sincères remerciements et ma profonde gratitude.

RÉSUMÉ

Ce rapport présente le projet de stage **SmartTrans-AI**, plateforme de gestion de transport enrichie par l'intelligence artificielle. L'application permet de planifier et de suivre des missions, de gérer les clients, les chauffeurs et les véhicules, et d'alerter sur les échéances critiques (permis de conduire, assurance, visite technique). Trois profils sont distingués : administrateur, responsable transport et chauffeur. L'authentification repose sur JSON Web Token (JWT) et des rôles. Un journal d'activités, en lecture seule, trace automatiquement les opérations. Un assistant conversationnel, ancré sur les données métier, répond aux questions opérationnelles et propose des ressources disponibles. La réalisation s'appuie sur Spring Boot (API REST), PostgreSQL, React et Vite. Ce document décrit le cadre du stage, l'analyse et la conception (cas d'utilisation, classes), la réalisation, les tests et les perspectives.

Mots-clés : gestion de transport, missions, JWT, Spring Boot, React, assistant IA, audit.

ABSTRACT

This internship report presents the **SmartTrans-AI** project, a transport management platform enhanced with artificial intelligence. The application supports the planning and tracking of missions, the management of clients, drivers and vehicles, and alerts on critical expiry dates (driving licence, insurance, technical inspection). Three user profiles are distinguished : administrator, transport manager and driver. Authentication is based on JSON Web Token (JWT) and roles. A read-only activity log automatically records operations. A conversational assistant, grounded in business data, answers operational questions and suggests available resources. The implementation relies on Spring Boot (REST API), PostgreSQL, React and Vite. This document describes the internship context, the analysis and design (use cases, class diagram), the implementation, the tests and the outlook.

Keywords : transport management, missions, JWT, Spring Boot, React, AI assistant, audit.

Table des matières

Liste des abréviations **8**

Liste des figures **9**

Liste des tableaux **10**

Liste des schémas **11**

Introduction générale **12**

1 Cadre du stage et étude préalable **13**

I Introduction . . . 13
II Organisme d'accueil et missions du stagiaire . . . 13
II.1 Présentation de l'organisme . . . 13
II.2 Missions confiées et planning . . . 13
III Problématique et besoins métier . . . 13
III.1 Problématique . . . 13
III.2 Besoins fonctionnels . . . 14
III.3 Besoins non fonctionnels . . . 14
IV Acteurs . . . 14
V Choix technologiques . . . 14
VI Conclusion . . . 15

2 État de l'art et positionnement **16**

I Introduction . . . 16
II Gestion de transport et systèmes d'information . . . 16
II.1 Le métier de TMS . . . 16
II.2 Limites des pratiques non intégrées . . . 16
III Authentification, API et applications web . . . 16
IV Intelligence artificielle en exploitation . . . 17
V Positionnement de SmartTrans-AI . . . 17
VI Conclusion . . . 17

3 Analyse et conception **18**

I Introduction . . . 18
II Cas d'utilisation . . . 18
II.1 Présentation du diagramme . . . 18
II.2 Description des cas principaux . . . 18
II.3 Fiches de cas d'utilisation . . . 18
III Scénarios dynamiques . . . 20
III.1 Authentification . . . 20
III.2 Cycle de vie d'une mission . . . 20
III.3 Question à l'assistant . . . 20
IV Diagramme de classes . . . 22
IV.1 Entités du domaine . . . 22
V Modèle de données . . . 23
VI Architecture logicielle . . . 23
VII Conclusion . . . 23

4 Réalisation **24**
I Introduction . . . 24
II Authentification et sécurité . . . 24
III Gestion des utilisateurs et des rôles . . . 24
IV Clients, chauffeurs et véhicules . . . 24
IV.1 Clients . . . 24
IV.2 Chauffeurs . . . 24
IV.3 Véhicules . . . 24
V Missions . . . 25
VI Tableau de bord, alertes et journal . . . 25
VII Assistant IA . . . 25
VIII Organisation du code et API . . . 25
IX Interface utilisateur . . . 26
X Conclusion . . . 26

5 Tests, déploiement et discussion **27**
I Introduction . . . 27
II Stratégie de tests . . . 27
II.1 Scénarios de recette . . . 27
III Déploiement . . . 27
IV Difficultés et solutions . . . 28
V Limites et perspectives . . . 28
VI Conclusion . . . 28

6 Conduite de projet **29**

I Introduction 29
II Méthode de travail 29
III Planning prévisionnel 29
IV Risques et parades 30
V Livrables 30
VI Conclusion 30

7 Présentation des interfaces **31**
I Introduction 31
II Connexion 31
III Tableau de bord 31
IV Gestion des chauffeurs 32
V Gestion des véhicules 34
VI Gestion des clients 34
VII Planification et suivi des missions 36
VIII Assistant 39
IX Journal d'activités 39
X Administration : utilisateurs et rôles 39
XI Conclusion 39

Conclusion générale et perspectives **42**

Références bibliographiques **43**

Annexe A — Guide technique de déploiement et d'exécution **44**

Annexe B — Documentation de l'API REST **46**

LISTE DES ABRÉVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CT	Contrôle technique (visite technique)
ENSIASD	École Nationale Supérieure d'Intelligence Artificielle et Sciences des Données de Taroudant
HTTP	HyperText Transfer Protocol
IA	Intelligence artificielle
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
REST	Representational State Transfer
SI	Système d'information
UML	Unified Modeling Language
UUID	Universally Unique Identifier

Table des figures

3.1 Diagramme de cas d'utilisation de SmartTrans-AI 19
3.2 Diagramme de séquence de la question à l'assistant IA 21
3.3 Diagramme de classes du domaine SmartTrans-AI 22

7.1 Écran de connexion SmartTrans-AI 31
7.2 Tableau de bord (profil administrateur) 32
7.3 Liste des chauffeurs 32
7.4 Formulaire de création d'un chauffeur 33
7.5 Fiche détail d'un chauffeur (alerte permis) 33
7.6 Formulaire de modification d'un chauffeur 34
7.7 Liste des véhicules 34
7.8 Formulaire de création d'un véhicule 35
7.9 Fiche détail d'un véhicule 35
7.10 Liste des clients 36
7.11 Formulaire de création d'un client 36
7.12 Fiche détail d'un client 37
7.13 Liste des missions 37
7.14 Formulaire de création d'une mission 38
7.15 Fiche détail d'une mission 38
7.16 Assistant conversationnel ancré sur les données métier 39
7.17 Journal d'activités (audit automatique) 40
7.18 Liste des utilisateurs 40
7.19 Formulaire de création d'un utilisateur 41
7.20 Catalogue des rôles 41

Liste des tableaux

1.1 Acteurs et périmètre fonctionnel 14

2.1 Positionnement des choix techniques 17

3.1 Cas d'utilisation : s'authentifier 19
3.2 Cas d'utilisation : planifier et suivre une mission 19
3.3 Cas d'utilisation : espace chauffeur 20
3.4 Principales relations du schéma 23

4.1 Extrait des ressources REST 25

5.1 Grille de recette fonctionnelle (extrait) 27

6.1 Découpage du projet en lots 29
6.2 Risques principaux et parades 30

2.1 Authentification 46
2.2 Utilisateurs et rôles 46
2.3 Clients, chauffeurs et véhicules 47
2.4 Missions 47
2.5 Journal et assistant 48

Liste des schémas

3.1 Architecture logique de SmartTrans-AI 23

INTRODUCTION GÉNÉRALE

Le secteur du transport routier de marchandises repose sur une coordination permanente entre clients, chauffeurs, véhicules et missions. Une organisation qui s'appuie uniquement sur des tableurs ou des échanges informels expose l'exploitant à des oublis : mission mal affectée, véhicule indisponible, permis ou assurance arrivés à échéance. Ces ruptures ont un coût opérationnel et réglementaire.

Le projet de stage **SmartTrans-AI** vise à concevoir et réaliser une application web de gestion de transport, enrichie par un assistant conversationnel. L'objectif n'est pas de remplacer le responsable d'exploitation, mais de centraliser les données, de tracer les actions et d'accélérer les recherches du type : combien de missions sont en cours, quel chauffeur est disponible, quel véhicule peut être affecté.

Le périmètre retenu comprend l'authentification, le tableau de bord, la gestion des utilisateurs (administrateur), des clients, des chauffeurs et des véhicules, le cycle de vie des missions, le journal d'activités, les alertes d'expiration et l'assistant IA. Sont volontairement exclus du produit livré un module documentaire de type GED et un écran de notifications push. Le chauffeur peut consulter ses missions, les démarrer et les terminer ; il ne peut pas les annuler.

Ce rapport s'organise en sept chapitres. Le premier situe le cadre du stage chez S. WATRANS. Le deuxième dresse un état de l'art et positionne le produit. Le troisième présente l'analyse et la conception (cas d'utilisation, classes, données). Le quatrième décrit la réalisation. Le cinquième traite des tests, du déploiement et des limites. Le sixième revient sur la conduite de projet. Le septième présente les interfaces (captures d'écran). Une conclusion générale ouvre des perspectives.

CHAPITRE 1 : CADRE DU STAGE ET ÉTUDE PRÉALABLE

I. Introduction

Ce chapitre précise le contexte du stage, les missions confiées, les acteurs métier et les choix technologiques qui ont orienté la réalisation de SmartTrans-AI.

II. Organisme d'accueil et missions du stagiaire

II.1 Présentation de l'organisme

S. WATRANS est une société spécialisée dans le transport de marchandises, la vente et l'achat de matériaux de construction, ainsi que dans la réalisation de travaux divers et de construction. Grâce à la diversité de ses activités, l'entreprise assure différents services liés au transport, à la logistique et au secteur du bâtiment.

Afin d'accompagner et de faciliter la gestion de ses différentes activités, la société dispose également d'un département informatique, chargé de la gestion et du développement des solutions informatiques utilisées au sein de l'entreprise. C'est au sein de ce département que j'ai effectué mon stage, en participant aux activités et aux travaux liés au développement et à l'amélioration des solutions informatiques de l'entreprise.

II.2 Missions confiées et planning

Les travaux confiés au stagiaire, au sein du département informatique, ont porté sur la spécification des rôles, la conception du modèle de données, le développement de l'API REST, l'interface React, l'assistant ancré sur les données, et la rédaction de ce rapport. Le découpage temporel et les risques sont détaillés au chapitre consacré à la conduite de projet.

III. Problématique et besoins métier

III.1 Problématique

Sans système partagé, le responsable transport ne dispose pas d'une vision fiable des missions (planifiées, en cours, terminées, annulées) ni d'alertes sur les documents arrivant à expiration. Le

chauffeur, de son côté, a besoin d'un accès simple à « ses » missions, sans droits de planification.

III.2 Besoins fonctionnels

Les besoins fonctionnels principaux sont : s'authentifier ; consulter un tableau de bord ; gérer les clients, chauffeurs, véhicules et missions ; consulter un journal d'audit ; interroger un assistant ; pour le chauffeur, suivre et clôturer ses courses.

III.3 Besoins non fonctionnels

L'application doit être utilisable via un navigateur, reposer sur une API documentée, séparer les droits selon le rôle, et rester déployable localement (développement) ou via des conteneurs. La confidentialité des mots de passe impose un hachage côté serveur. L'assistant ne doit pas inventer des missions absentes de la base : les réponses s'appuient sur un contexte métier extrait de PostgreSQL.

IV. Acteurs

Trois acteurs correspondent aux codes de rôle `ADMIN`, `TRANSPORT_MANAGER` et `DRIVER` (tableau 1.1).

Tableau : 1.1 : Acteurs et périmètre fonctionnel

Acteur	Périmètre
Administrateur	Utilisateurs et rôles, tout le métier, journal, assistant, tableau de bord
Responsable transport	Métier, journal, assistant, tableau de bord ; pas de gestion des comptes utilisateurs
Chauffeur	Tableau de bord, ses missions, démarrage et fin de mission, profil, alertes permis

V. Choix technologiques

L'architecture retenue est une application web à deux volets : une API REST côté serveur et une interface web côté client. Chaque composant de la stack a un rôle précis, rappelé ci-dessous, avec son logo à droite.

Java. Java est le socle du backend : typage statique et code structuré, testable pendant le stage.



Spring Boot. Spring Boot orchestre l'API REST (`/api/v1`), JPA, la sécurité et les tests [1].



PostgreSQL. PostgreSQL stocke utilisateurs, clients, chauffeurs, véhicules, missions et journal [3].



JWT. Après le login, le client envoie un JSON Web Token [5] dans `Authorization`. L'API reste sans état.

React. React construit l'interface (tableau de bord, formulaires, vues selon le profil) [2].

TypeScript. TypeScript type le client et limite les écarts de contrat avec l'API (identifiants, statuts, rôles).

Vite. Vite compile le front. Un proxy vers le port 8080 évite les blocages CORS en développement.

Flyway. Flyway versionne le schéma : les migrations alignent la base sur le code, en local comme en conteneur.

OpenAI. Avec `OPENAI_API_KEY`, l'assistant appelle Chat Completions (`gpt-4o-mini`). Sinon, le mode `GROUNDED` s'appuie sur les données locales.

VI. Conclusion

Le cadre du stage fixe un produit de transport, trois rôles et une architecture API plus interface. Le chapitre suivant situe ce choix face à l'état de l'art.

CHAPITRE 2 : ÉTAT DE L'ART ET POSITIONNEMENT

I. Introduction

Avant de spécifier SmartTrans-AI, il convient de situer le projet dans le paysage des systèmes de gestion de transport (TMS), des pratiques d'exploitation encore largement manuelles, et des usages émergents de l'intelligence artificielle en logistique. Ce chapitre justifie le positionnement du livrable : un outil métier ciblé, sécurisé, et un assistant ancré sur les données de l'entreprise plutôt qu'un TMS générique du marché.

II. Gestion de transport et systèmes d'information

II.1 Le métier de TMS

Un *Transport Management System* vise à planifier les tournées, affecter les ressources (chauffeurs, véhicules), suivre l'exécution et produire des indicateurs. Les éditeurs internationaux proposent des suites souvent coûteuses, longues à paramétrer, et parfois surdimensionnées pour une PME de transport et de BTP telle que S. WATRANS. À l'opposé, de nombreuses exploitations s'appuient encore sur des tableurs, la messagerie et le téléphone. Ces outils restent souples, mais ils ne garantissent ni l'unicité de l'information, ni la traçabilité, ni le contrôle d'accès par rôle.

II.2 Limites des pratiques non intégrées

Lorsque les missions sont saisies dans un fichier partagé, les risques sont connus : versions divergentes, absence d'historique fiable, difficulté à savoir si un chauffeur est déjà en course, oubli des dates de permis ou d'assurance. Le besoin exprimé pendant le stage n'était pas de reproduire toute la richesse d'un TMS du marché, mais de couvrir un noyau : clients, flotte, missions, alertes, journal, et un canal de questionnement rapide (assistant).

III. Authentification, API et applications web

Les applications d'entreprise actuelles séparent souvent une interface web d'une API REST. L'authentification par session serveur reste possible ; le jeton JWT, largement documenté [5], permet à l'interface de rester sans état côté API, ce qui simplifie le déploiement et le passage à l'échelle. Spring Security et Spring Boot offrent un cadre éprouvé pour exposer des ressources protégées par rôle [1]. Côté client, React s'est imposé pour des interfaces modulaires [2]. PostgreSQL reste un choix robuste pour un modèle relationnel métier [3].

Le tableau 2.1 résume les alternatives envisagées et le choix retenu.

Tableau : 2.1 : Positionnement des choix techniques

Composant	Alternatives	Choix et motif
Backend	Node.js, Django, .NET	Spring Boot : typage Java, sécurité, JPA, tests d'intégration
Frontend	Angular, Vue	React + Vite : écosystème, rapidité de développement
Base de données	MySQL, MongoDB	PostgreSQL : relations, contraintes, JSON pour l'audit
Auth	Session, OAuth2 complet	JWT : API stateless, rôles, simplicité de mise en œuvre
IA	Chat générique sans contexte	Assistant ancré sur missions, chauffeurs, véhicules, clients

IV. Intelligence artificielle en exploitation

Les modèles de langage permettent de formuler des questions en langue naturelle. Sans ancrage, ils hallucinent des effectifs ou des immatriculations. La littérature et les retours d'usage insistent sur la *grounding* : fournir un contexte issu du SI. SmartTrans-AI adopte cette approche : extraction d'un résumé opérationnel, puis génération éventuelle via une clé d'API, ou réponses déterministes si la clé est absente. L'IA n'est pas un module de planification automatique des tournées (problème d'optimisation combinatoire) ; elle assiste la consultation et la suggestion de ressources disponibles.

V. Positionnement de SmartTrans-AI

SmartTrans-AI se situe entre le tableur et le TMS complet. Il est spécifique au besoin du département informatique de S. WATRANS : digitaliser le suivi des missions et de la conformité minimale de la flotte, avec trois profils, un audit automatique et un assistant. Les fonctionnalités de GED et de notifications temps réel ont été écartées pour tenir le calendrier du stage et rester sur un produit démontrable.

VI. Conclusion

L'état de l'art confirme l'intérêt d'une application web sécurisée, d'un modèle relationnel clair, et d'une IA contrainte par les données. Le chapitre suivant traduit ces orientations en cas d'utilisation et en conception UML.

CHAPITRE 3 : ANALYSE ET CONCEPTION

I. Introduction

Ce chapitre formalise le système : cas d'utilisation, scénarios dynamiques (dont le diagramme de séquence de l'assistant), diagramme de classes du domaine, schéma relationnel et architecture de déploiement. Les diagrammes UML suivent le guide ENSIASD pour la numérotation des figures [4].

II. Cas d'utilisation

II.1 Présentation du diagramme

Le diagramme de cas d'utilisation (figure 3.1) regroupe les fonctions autour de quatre paquets : commun, métier, administration et espace chauffeur. Tous les acteurs s'authentifient par JWT. L'administrateur et le responsable partagent la gestion des clients, chauffeurs, véhicules et missions, ainsi que le journal et l'assistant. Seul l'administrateur gère les utilisateurs et les rôles. Le chauffeur consulte ses missions, change le statut (démarrer / terminer) et met à jour son profil.

II.2 Description des cas principaux

L'authentification délivre un jeton utilisé sur les routes protégées. Le tableau de bord agrège des indicateurs (volumes de missions, graphiques, rappels d'expiration). La planification d'une mission associe un client obligatoire, un chauffeur et un véhicule optionnels, un trajet, une date et un statut (`PLANIFIED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`). Le journal est alimenté automatiquement (créations, mises à jour, suppressions, changements de statut, connexions) et n'offre pas de CRUD manuel. L'assistant répond à des questions du type effectifs de missions ou suggestions de chauffeur et de véhicule disponibles.

II.3 Fiches de cas d'utilisation

Les tableaux 3.1 à 3.3 décrivent les cas les plus structurants, au format habituel (acteur, préconditions, enchaînement, exceptions).

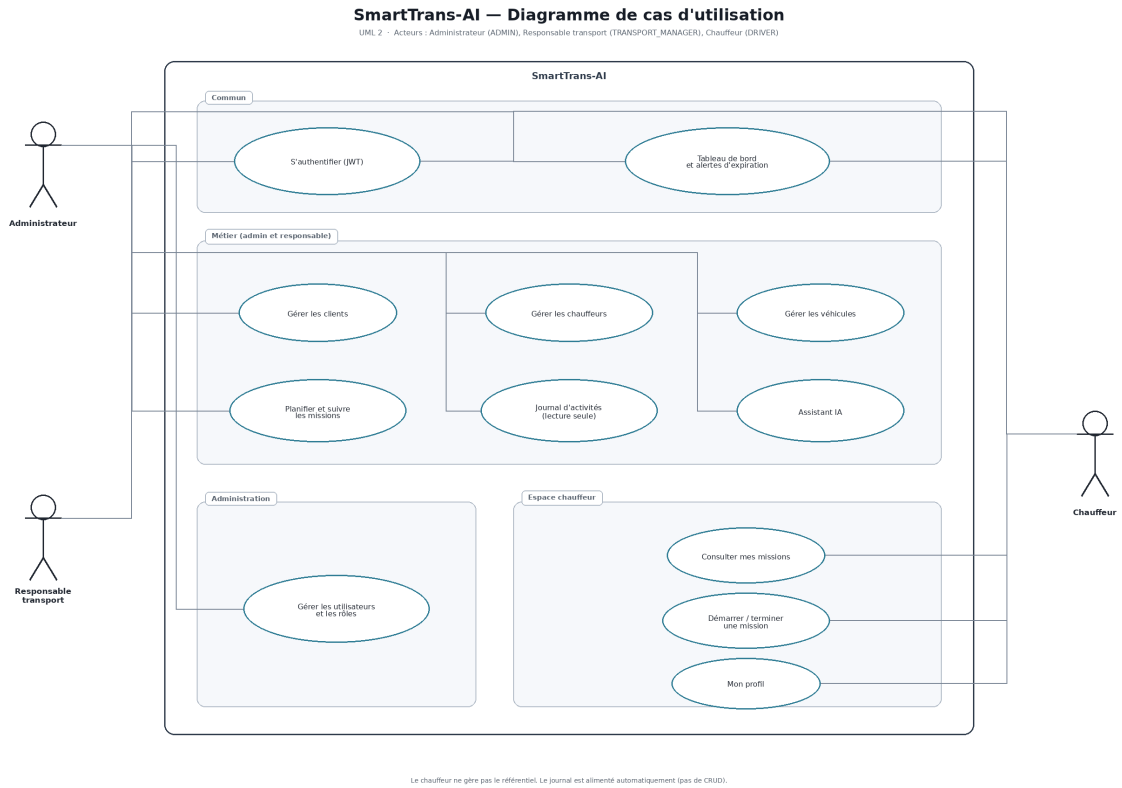


Figure : 3.1 : Diagramme de cas d'utilisation de SmartTrans-AI

Tableau : 3.1 : Cas d'utilisation : s'authentifier

Identifiant	CU-AUTH
Acteurs	Administrateur, responsable transport, chauffeur
Précondition	Compte existant, activé
Enchaînement	Saisie courriel et mot de passe ; vérification du hachage ; émission d'un JWT ; redirection vers le tableau de bord
Exceptions	Identifiants invalides, compte désactivé, origine CORS refusée
Postcondition	Le client conserve le jeton pour les appels suivants

Tableau : 3.2 : Cas d'utilisation : planifier et suivre une mission

Identifiant	CU-MIS
Acteurs	Administrateur, responsable transport (écriture) ; chauffeur (statut de ses missions)
Précondition	Client existant ; éventuellement chauffeur et véhicule disponibles
Enchaînement	Création (référence, trajet, date) ; affectation ; passage planifiée → en cours → terminée ; consultation des listes et filtres
Exceptions	Référence déjà utilisée ; chauffeur non affecté qui tente un changement de statut ; tentative d'annulation par le chauffeur
Postcondition	Mission persistée ; événement d'audit enregistré

Tableau : 3.3 : Cas d'utilisation : espace chauffeur

Identifiant	CU-DRV
Acteur	Chauffeur
Précondition	Compte lié à une fiche chauffeur
Enchaînement	Consultation de mes missions ; démarrage ; clôture ; mise à jour du profil et de la disponibilité
Exceptions	Mission non affectée ; statut métier incompatible

III. Scénarios dynamiques

III.1 Authentification

Le navigateur envoie les identifiants à `POST /api/v1/auth/login` sans en-tête Bearer. L'API valide les informations, construit le jeton et renvoie le profil (rôles). Les appels suivants portent `Authorization: Bearer`. Un filtre ignore les routes d'authentification afin d'éviter un 403 sur le login.

III.2 Cycle de vie d'une mission

Le responsable crée la mission (statut planifiée). Le chauffeur authentifié appelle le changement de statut. Le serveur vérifie l'affectation et la transition autorisée, met à jour les horodatages `startedAt` / `completedAt`, et journalise l'action. L'annulation n'est pas exposée au profil chauffeur.

III.3 Question à l'assistant

L'administrateur ou le responsable transport saisit une question depuis l'interface. Le diagramme de séquence de la figure 3.2 détaille les échanges. Le navigateur envoie `POST /api/v1/assistant/ask` avec l'en-tête `Authorization: Bearer`. Le filtre JWT vérifie la signature et l'expiration du jeton. S'il est absent, invalide ou expiré, l'API répond `401` et l'utilisateur revient à l'écran de connexion, sans lecture des données.

Lorsque le jeton est valide, `AssistantService` interroge PostgreSQL et constitue un contexte ancré (missions, chauffeurs, véhicules, clients). Si une clé d'API est configurée, ce contexte est transmis à OpenAI (`POST /chat/completions`) et la réponse porte le mode `LLM`. Sinon, ou si l'appel échoue, le service compose lui-même une réponse courte, par exemple un effectif de missions (mode `GROUNDED`). Dans les deux cas, le client reçoit le texte, le mode et les citations, puis les affiche.

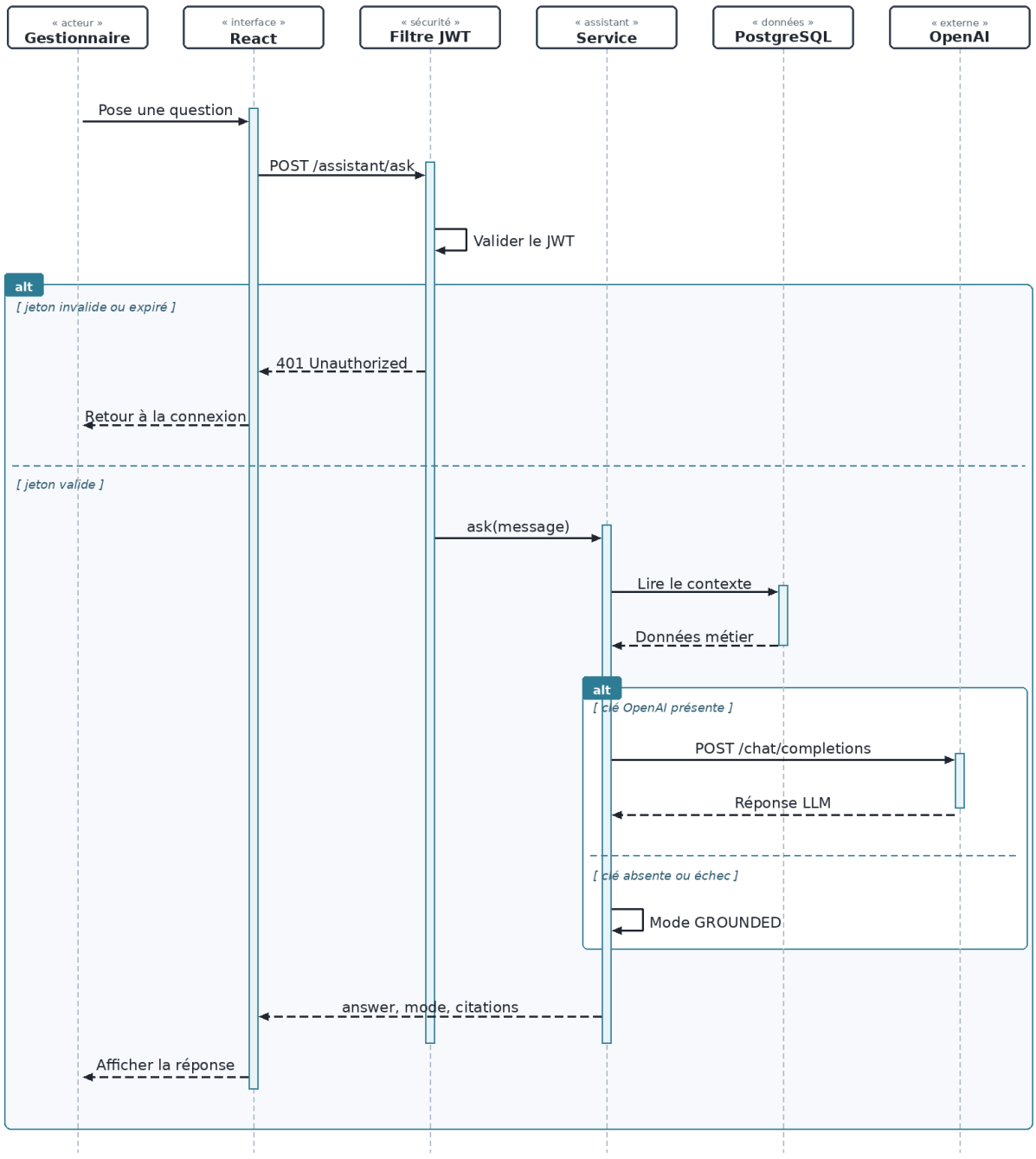


Figure : 3.2 : Diagramme de séquence de la question à l'assistant IA

IV. Diagramme de classes

IV.1 Entités du domaine

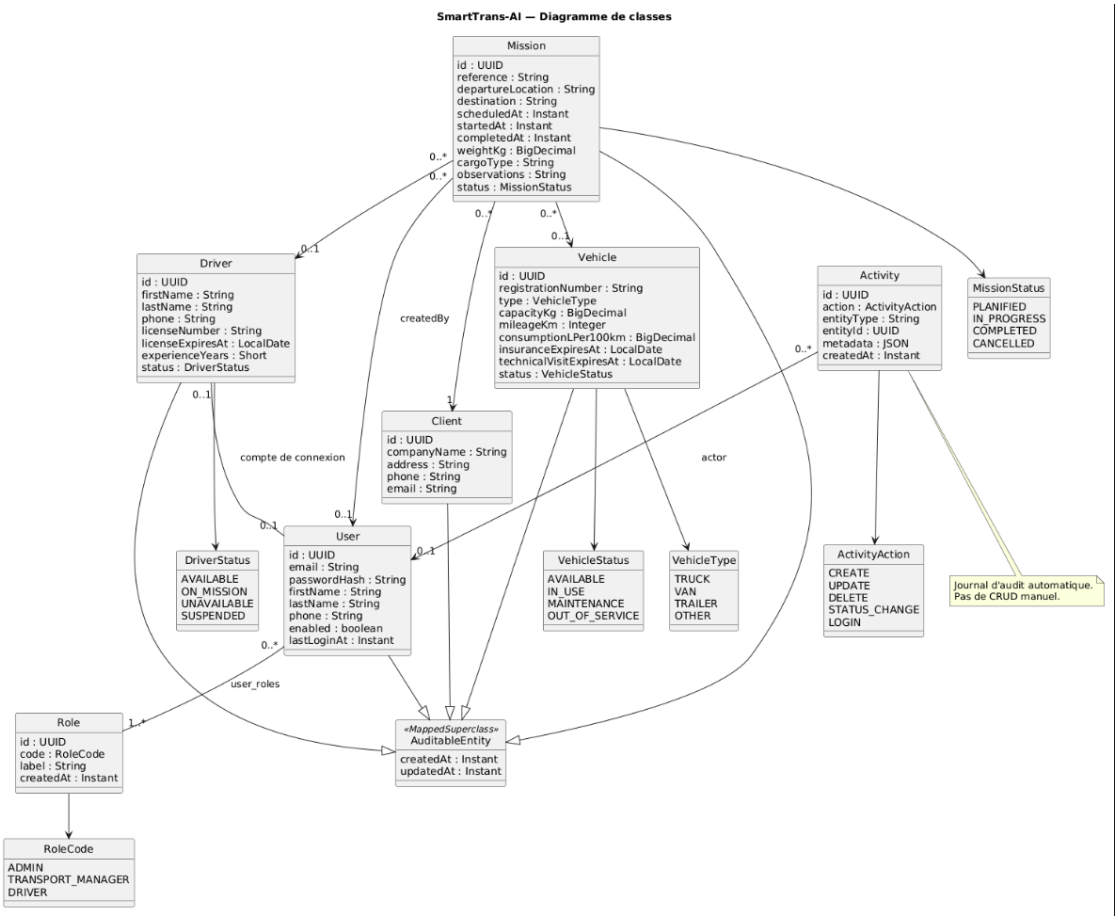


Figure : 3.3 : Diagramme de classes du domaine SmartTrans-AI

Le diagramme de la figure 3.3 représente le domaine persisté de SmartTrans-AI. Au centre, la classe `Mission` matérialise une course : référence, trajet, dates, tonnage et statut. Elle dépend d'un `Client` obligatoire et, le cas échéant, d'un `Driver` et d'un `Vehicle` ; l'association `createdBy` mémorise l'utilisateur qui a planifié la course. `User` et `Role` sont reliés par une association plusieurs-à-plusieurs (`user_roles`), ce qui permet d'attribuer un ou plusieurs profils (administrateur, responsable transport, chauffeur). Un `Driver` peut être lié à au plus un compte `User`, afin que le chauffeur se connecte et suive uniquement ses missions. `Activity` constitue le journal d'audit : chaque action (création, mise à jour, suppression, changement de statut, connexion) pointe vers l'acteur. Les classes métier héritent de `AuditableEntity` (`createdAt`, `updatedAt`). Les énumérations `RoleCode`, `DriverStatus`, `VehicleStatus`, `VehicleType`, `MissionStatus` et `ActivityAction` limitent les valeurs autorisées. Les modules Document et Notification, hors périmètre livré, n'apparaissent pas sur ce diagramme.

V. Modèle de données

Les tables PostgreSQL correspondent aux classes : `users`, `roles`, `user_roles`, `clients`, `drivers`, `vehicles`, `missions`, `activities`. Les identifiants sont des UUID. Le schéma est versionné par Flyway (validation au démarrage hors profil de test). Le tableau 3.4 synthétise les relations.

Tableau : 3.4 : Principales relations du schéma

Table	Lien	Cardinalité
`user_roles`	users – roles	n–n
`drivers.user_id`	drivers – users	0..1–0..1
`missions.client_id`	missions – clients	n–1
`missions.driver_id`	missions – drivers	n–0..1
`missions.vehicle_id`	missions – vehicles	n–0..1
`activities.actor_user_id`	activities – users	n–0..1

VI. Architecture logicielle

L'interface React s'exécute dans le navigateur. En développement, le serveur Vite (port 5173) transmet les appels `/api` vers Spring Boot (port 8080). En production conteneurisée, un serveur HTTP (Nginx) sert les fichiers statiques et reverse-proxifie l'API. PostgreSQL stocke le métier. Le schéma 3.1 résume les flux.

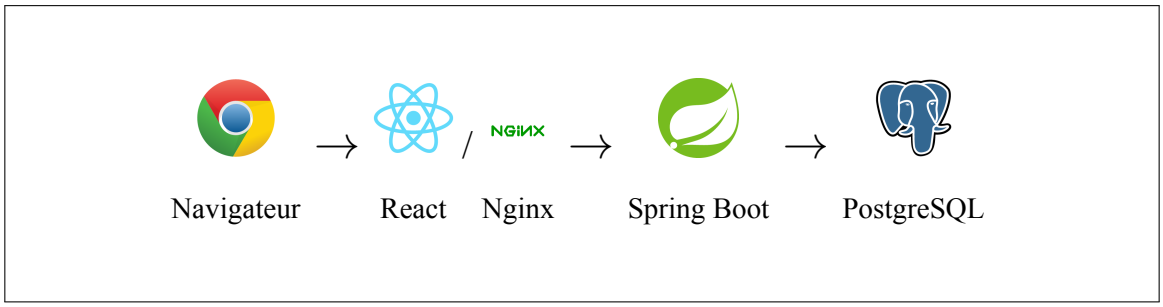


Schéma : 3.1 : Architecture logique de SmartTrans-AI

VII. Conclusion

L'analyse produit un système à trois acteurs, un domaine centré sur la mission, et une architecture présentation / API / données. Le chapitre suivant décrit la traduction de cette conception en logiciel.

CHAPITRE 4 : RÉALISATION

I. Introduction

Ce chapitre présente la mise en œuvre : sécurité, modules métier, interface selon le rôle, assistant et journal d'activités.

II. Authentification et sécurité

L'utilisateur s'identifie par courriel et mot de passe. Le serveur vérifie le mot de passe haché, génère un JWT et le client le transmet dans l'en-tête `Authorization`. Les routes `/api/v1/auth/` restent accessibles sans jeton. Le jeton suit le standard JWT [5]. La configuration CORS autorise les origines locales (`localhost` et `127.0.0.1`) afin que le front de développement puisse appeler l'API. Les contrôleurs restreignent les opérations selon les autorités Spring Security correspondant aux rôles.

III. Gestion des utilisateurs et des rôles

L'administrateur crée et met à jour les comptes, active ou désactive un utilisateur, et assigne un ou plusieurs rôles. Cette fonction n'est pas exposée au responsable transport ni au chauffeur, afin de limiter la surface d'administration.

IV. Clients, chauffeurs et véhicules

IV.1 Clients

Un client est identifié par le nom de l'entreprise, une adresse, un téléphone et un courriel optionnel. Il est obligatoire sur chaque mission.

IV.2 Chauffeurs

Le chauffeur porte un numéro de permis, une date d'expiration, une ancienneté et un statut (disponible, en mission, indisponible, suspendu). Le lien optionnel avec un `User` permet la connexion à l'espace chauffeur.

IV.3 Véhicules

Le véhicule est caractérisé par l'immatriculation, le type, la capacité, le kilométrage, des dates d'assurance et de visite technique, et un statut d'exploitation. Ces dates alimentent les rappels du tableau de bord.

V. Missions

La mission possède une référence unique, un trajet (départ, destination), une date planifiée, un tonnage et un type de marchandise optionnels. Le responsable et l'administrateur créent et modifient les champs. Le changement de statut du chauffeur passe par une ressource dédiée : il peut passer une mission qui lui est affectée de planifiée à en cours, puis à terminée. L'annulation reste du ressort de l'exploitation.

VI. Tableau de bord, alertes et journal

Le tableau de bord affiche des indicateurs, des graphiques (activité de la semaine, répartition des statuts), les prochaines missions et les rappels (permis, assurance, contrôle technique). Le journal d'activités liste les événements avec un libellé d'objet lisible (référence de mission, nom, immatriculation, entreprise, courriel) plutôt qu'un identifiant technique seul.

VII. Assistant IA

L'assistant est réservé à l'administrateur et au responsable transport. Il s'appuie sur un extrait des missions, chauffeurs, véhicules et clients. Lorsqu'une clé de modèle de langage est configurée, les réponses sont générées à partir de ce contexte ; sinon, des règles déterministes couvrent les questions fréquentes (effectifs, suggestions). L'objectif pédagogique et opérationnel est d'« ancrer » l'IA sur le SI, afin de limiter les hallucinations. Les échanges correspondants sont ceux de la figure 3.2.

VIII. Organisation du code et API

Le backend est structuré par packages métier (`auth`, `user`, `role`, `client`, `driver`, `vehicle`, `mission`, `activity`, assistant). Chaque module typique expose un contrôleur REST, un service et un dépôt JPA. Le préfixe public est `/api/v1`. Le tableau 4.1 donne un extrait des ressources.

Tableau : 4.1 : Extrait des ressources REST

Ressource	Méthode	Usage
`/auth/login`	POST	Authentification, JWT
`/users`	GET, POST, PATCH	Comptes (admin)
`/clients`, `/drivers`, `/vehicles`	CRUD	Référentiels
`/missions`	CRUD	Planification
`/missions/{id}/status`	PATCH	Statut (dont chauffeur)
`/activities`	GET	Journal lecture seule
`/assistant`	POST	Question / suggestions

Les écrans React (listes, formulaires, tableau de bord, assistant) consomment ces ressources via un client HTTP unique, qui attache le jeton sauf sur le login.

IX. Interface utilisateur

L'interface distingue la navigation : le chauffeur accède au tableau de bord, à ses missions et à son profil ; les autres rôles voient en plus les menus d'exploitation, le journal et l'assistant. Les formulaires de saisie et les listes correspondent aux ressources de l'API. Les messages d'erreur HTTP (par exemple un refus d'authentification) sont traduits pour l'utilisateur.

X. Conclusion

La réalisation couvre la chaîne authentification–métier–pilotage–assistant, avec une séparation claire des droits. Le chapitre suivant évalue la qualité logicielle et le déploiement.

CHAPITRE 5 : TESTS, DÉPLOIEMENT ET DISCUSSION

I. Introduction

Ce chapitre présente la stratégie de tests, les modalités de déploiement, les difficultés rencontrées et les limites du livrable.

II. Stratégie de tests

Le backend dispose de tests Maven. Le profil de test utilise H2 pour une partie des cas ; des tests d'intégration s'appuient sur PostgreSQL embarqué afin de valider Flyway et JPA. Des scénarios HTTP couvrent l'authentification JWT et des exemples de CRUD (utilisateurs). Cette base ne remplace pas une recette métier complète mais sécurise les régressions sur la sécurité et la persistance.

II.1 Scénarios de recette

Le tableau 5.1 propose une grille de recette manuelle, utilisable lors d'une démonstration à S. WATRANS.

Tableau : 5.1 : Grille de recette fonctionnelle (extrait)

Id	Scénario	Résultat attendu
R1	Login admin valide	Accès dashboard et menus d'admin
R2	Login chauffeur	Menus limités (missions, profil)
R3	Création mission + client	Mission planifiée, trace d'audit
R4	Chauffeur démarre puis termine	Statuts et dates mis à jour
R5	Chauffeur tente une annulation	Refus (droit insuffisant)
R6	Permis ou assurance proche	Alerte sur le tableau de bord
R7	Question « combien de missions »	Réponse courte, cohérente avec la base
R8	Login erroné	Message clair, pas d'accès

III. Déploiement

En développement, PostgreSQL peut être lancé via Docker Compose, l'API par Maven, l'interface par `npm run dev`. Une stack de production assemble base, API et Nginx (port d'écoute distinct, par exemple 8088). Les secrets (mot de passe de base, secret JWT, clé d'API éventuelle) doivent être fournis par variables d'environnement et non versionnés. Un compte administrateur peut être créé au premier démarrage si la base est vide (*bootstrap*), ce qui facilite

une démonstration.

IV. Difficultés et solutions

Plusieurs points ont demandé un diagnostic : autorisation CORS entre `localhost` et `127.0.0.1`, envoi d'un jeton Bearer sur la route de login, messages d'erreur 403 peu lisibles, alimentation du tableau de bord par des données de démonstration. Chaque correctif a visé un comportement prévisible pour l'utilisateur final (connexion depuis le front de développement, listes et graphiques non vides).

V. Limites et perspectives

Le livrable n'inclut pas de bus d'événements (Kafka), ni de notifications temps réel, ni de module documentaire. L'assistant dépend de la qualité du contexte injecté et, le cas échéant, d'un prestataire de modèle de langage. Un durcissement (HTTPS, secrets, politique de mots de passe, journalisation centralisée) reste souhaitable avant un usage en production réelle. Ces pistes peuvent faire l'objet d'un travail ultérieur.

VI. Conclusion

Les tests automatisés et le déploiement local ou conteneurisé rendent le projet démontrable. Les limites rappellent qu'il s'agit d'un produit de stage, extensible. Le chapitre suivant revient sur le planning, les risques et les livrables.

CHAPITRE 6 : CONDUITE DE PROJET

I. Introduction

La réalisation de SmartTrans-AI s'est inscrite dans une démarche itérative, compatible avec un stage en entreprise : lots fonctionnels successifs, recette interne, puis rédaction du rapport. Ce chapitre présente le découpage, les risques et les livrables.

II. Méthode de travail

Le travail a suivi un cycle proche de l'agilité légère : expression du besoin avec l'encadrante entreprise, maquettes et API, développement par module (authentification, référentiels, missions, tableau de bord, journal, assistant), puis ajustements (droits chauffeur, CORS, messages d'erreur). Les diagrammes UML ont servi de contrat entre l'analyse et le code. Les tests automatisés du backend ont été exécutés à chaque palier critique (sécurité, persistance).

III. Planning prévisionnel

Le tableau 6.1 présente le découpage du projet sur un mois de stage (quatre semaines). L'ordre des lots correspond à la dépendance technique (on n'expose pas les missions sans authentification).

Tableau : 6.1 : Découpage du projet en lots

Période	Lot	Résultat attendu
Semaine 1	Cadre, besoin, rôles, modèle de données, sécurité JWT	Cahier des charges interne, schéma Flyway, login
Semaine 2	Utilisateurs, clients, chauffeurs, véhicules, missions	CRUD protégés, cycle de vie des courses
Semaine 3	Tableau de bord, alertes, journal	Pilotage et traçabilité
Semaine 4	Assistant, droits chauffeur, recette, Docker, rapport ENSIASD	Espace chauffeur, démo et mémoire

IV. Risques et parades

Cinq risques ont été suivis pendant le stage. Le tableau 6.2 indique, pour chacun, l'impact redouté et la parade appliquée.

Tableau : 6.2 : Risques principaux et parades

Risque	Impact	Parade
Périmètre trop large (GED, Kafka, notifications)	Retard	Priorisation du noyau métier
Hallucinations de l'assistant	Perte de confiance	Contexte SQL, réponses courtes, mode sans clé
Blocage CORS / JWT en local	Impossibilité de se connecter	Origines multiples, exclusion des routes d'auth
Données de démo vides	Tableau de bord peu lisible	Jeux de missions et alertes de démonstration
Dépendance à un prestataire LLM	Coût, indisponibilité	Mode déterministe local

V. Livrables

Les livrables du stage comprennent : le code source (backend Spring Boot, frontend React), le schéma de base, la documentation OpenAPI, les diagrammes UML, le présent rapport, et une procédure de démarrage local. Le produit reste un prototype d'entreprise, destiné à être repris par le département informatique.

VI. Conclusion

La conduite de projet a privilégié un noyau démontrable plutôt qu'une couverture exhaustive du marché TMS. Cette discipline a permis d'aboutir à une application cohérente, testable et documentée. Le chapitre suivant présente les interfaces réalisées, écran par écran.

CHAPITRE 7 : PRÉSENTATION DES INTERFACES

I. Introduction

Ce chapitre illustre le livrable tel qu'il apparaît à l'utilisateur. Les captures ont été réalisées sur l'environnement local (`http://127.0.0.1:5173`), avec le compte administrateur. Elles correspondent aux écrans du menu principal : connexion, tableau de bord, référentiels, missions, assistant, journal d'activités et administration. L'espace « Mon profil » du chauffeur n'est pas reproduit ici : il n'est accessible qu'avec un compte `DRIVER`.

II. Connexion

La figure 7.1 montre l'écran public de connexion. L'utilisateur saisit son courriel et son mot de passe. Un bouton unique déclenche l'appel d'authentification. En cas de succès, un jeton JWT est stocké côté client et l'application redirige vers le tableau de bord.

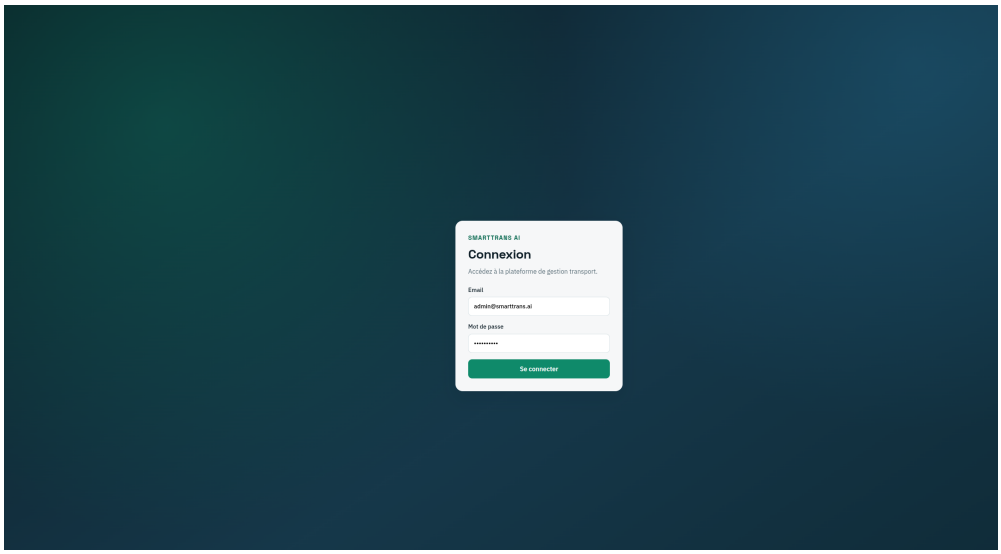


Figure : 7.1 : Écran de connexion SmartTrans-AI

III. Tableau de bord

La figure 7.2 présente la vue de pilotage. Les cartes du haut résument les volumes de missions, les ressources disponibles et les alertes (permis, assurance, visite technique). Les graphiques visualisent l'activité de la semaine et la répartition des statuts. Les listes du bas rappellent les prochaines courses, les chauffeurs en mission et les échéances.

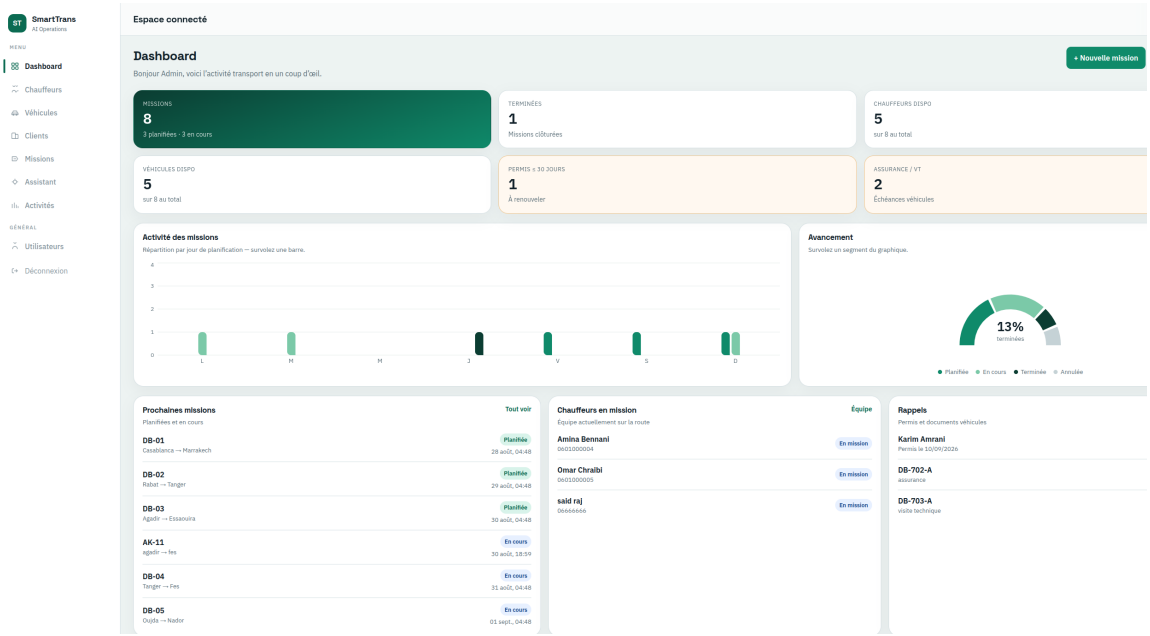


Figure : 7.2 : Tableau de bord (profil administrateur)

IV. Gestion des chauffeurs

La liste (figure 7.3) permet de rechercher un chauffeur, de filtrer par statut et par échéance de permis. La création (figure 7.4) saisit l'identité, le permis et le statut. Le détail (figure 7.5) affiche les rappels d'expiration. La modification (figure 7.6) reprend le même formulaire prérempli.

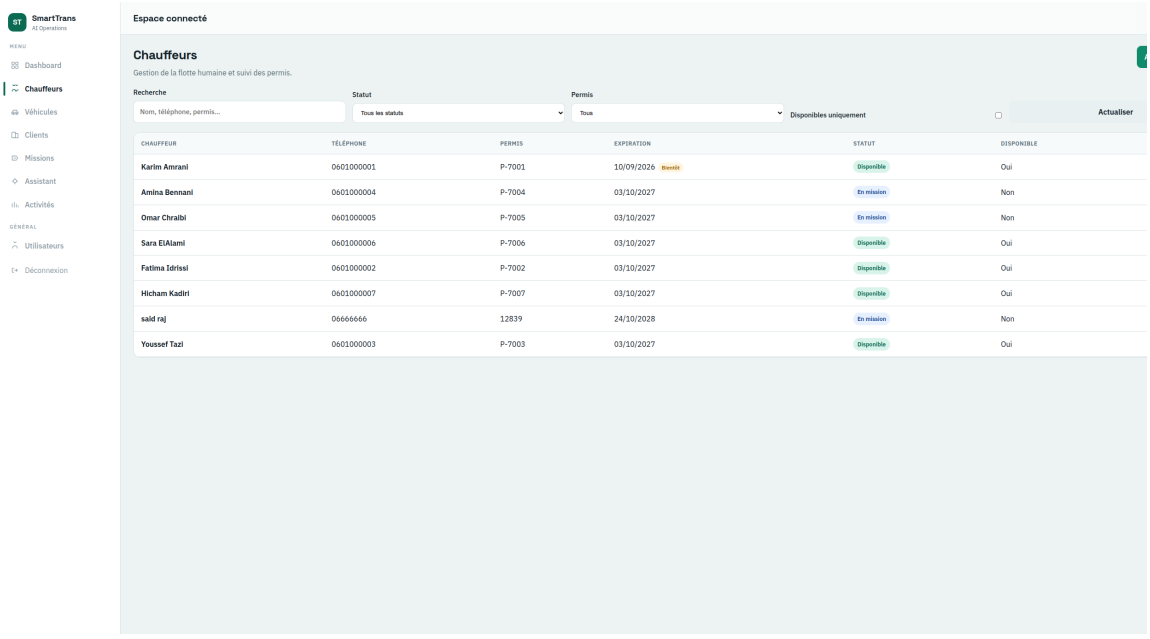


Figure : 7.3 : Liste des chauffeurs

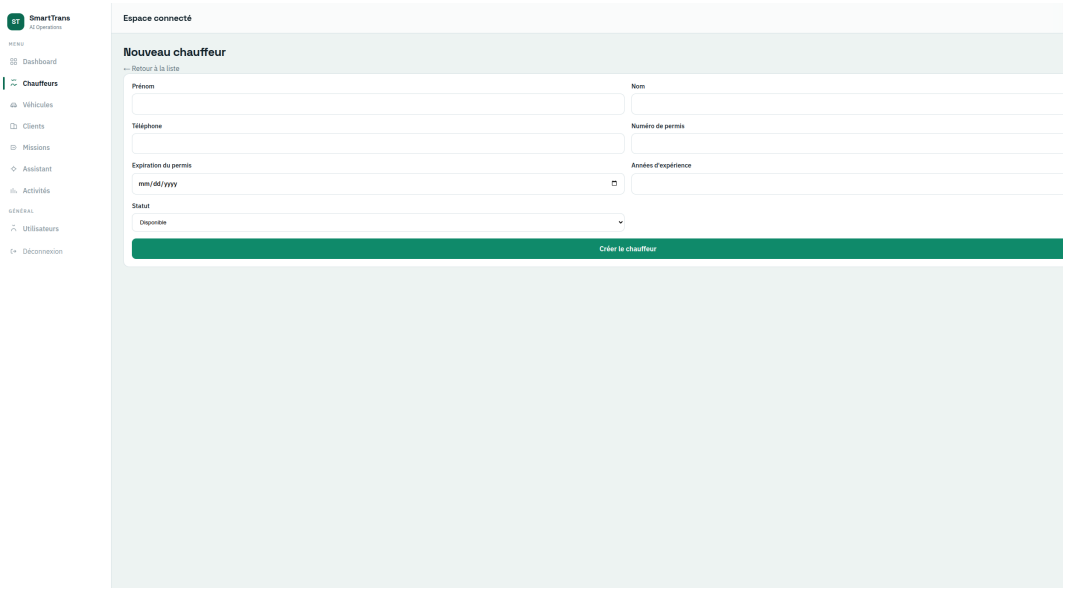


Figure : 7.4 : Formulaire de création d'un chauffeur

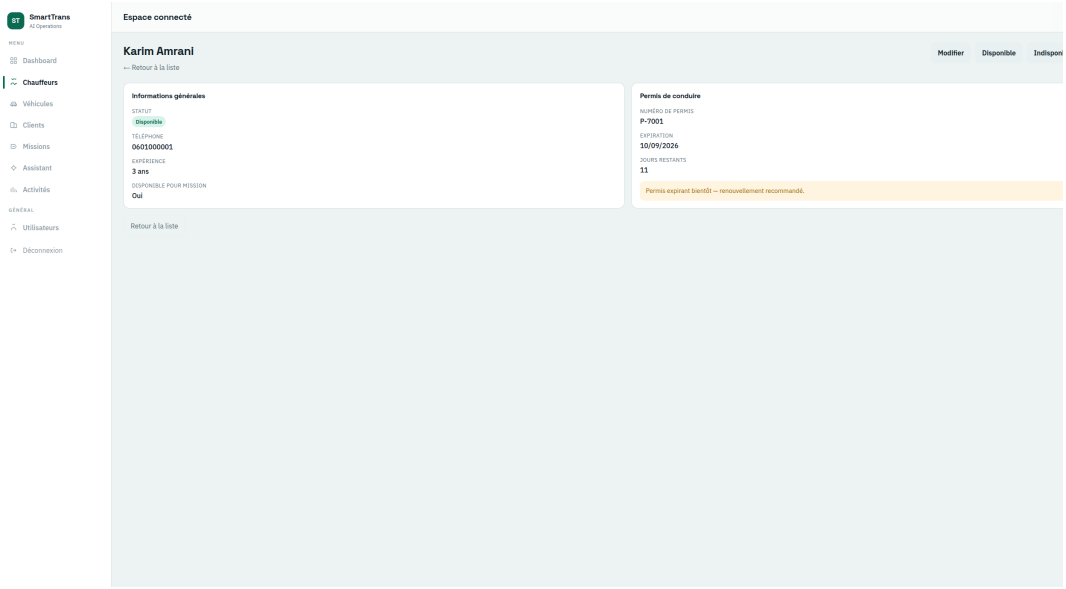


Figure : 7.5 : Fiche détail d'un chauffeur (alerte permis)

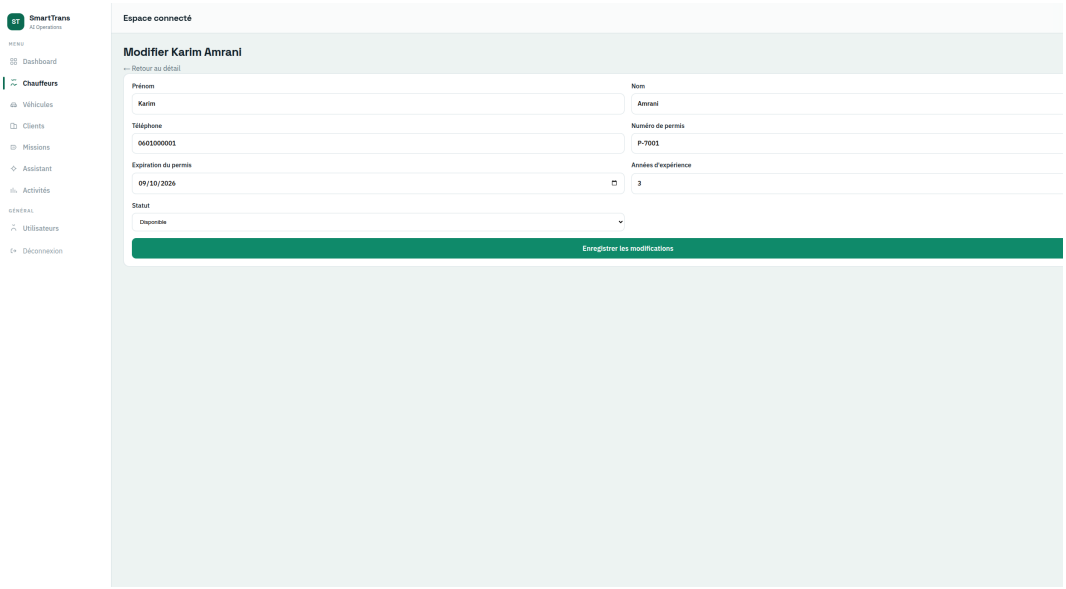


Figure : 7.6 : Formulaire de modification d'un chauffeur

V. Gestion des véhicules

La liste (figure 7.7) suit immatriculation, type, kilométrage, dates d'assurance et de visite technique, et statut d'exploitation. Le formulaire de création (figure 7.8) et la fiche détail (figure 7.9) reprennent ces champs, avec un indicateur de validité des documents.

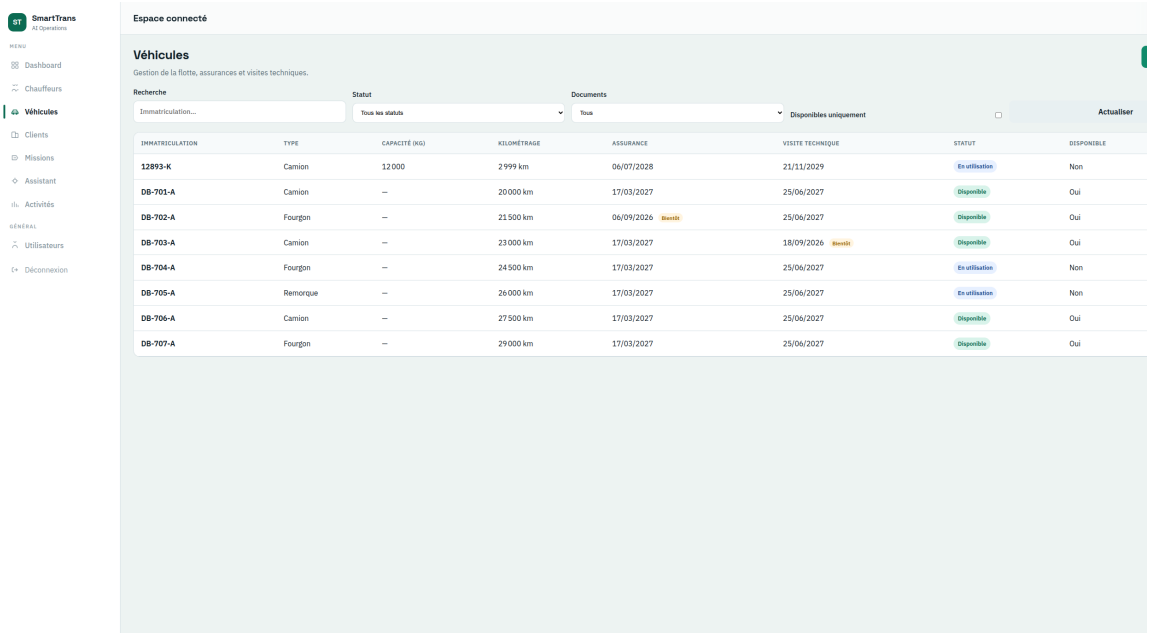


Figure : 7.7 : Liste des véhicules

VI. Gestion des clients

Les clients sont les entreprises destinataires des missions. La liste (figure 7.10) affiche le nom de l'entreprise, les coordonnées et l'adresse. La création (figure 7.11) et le détail (figure 7.12)

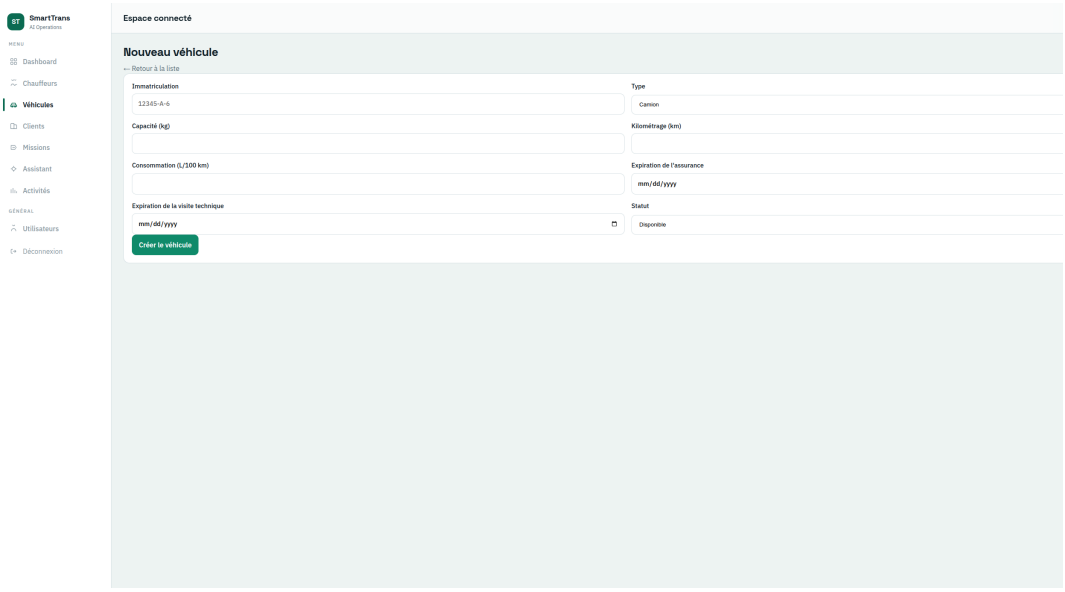


Figure : 7.8 : Formulaire de création d'un véhicule

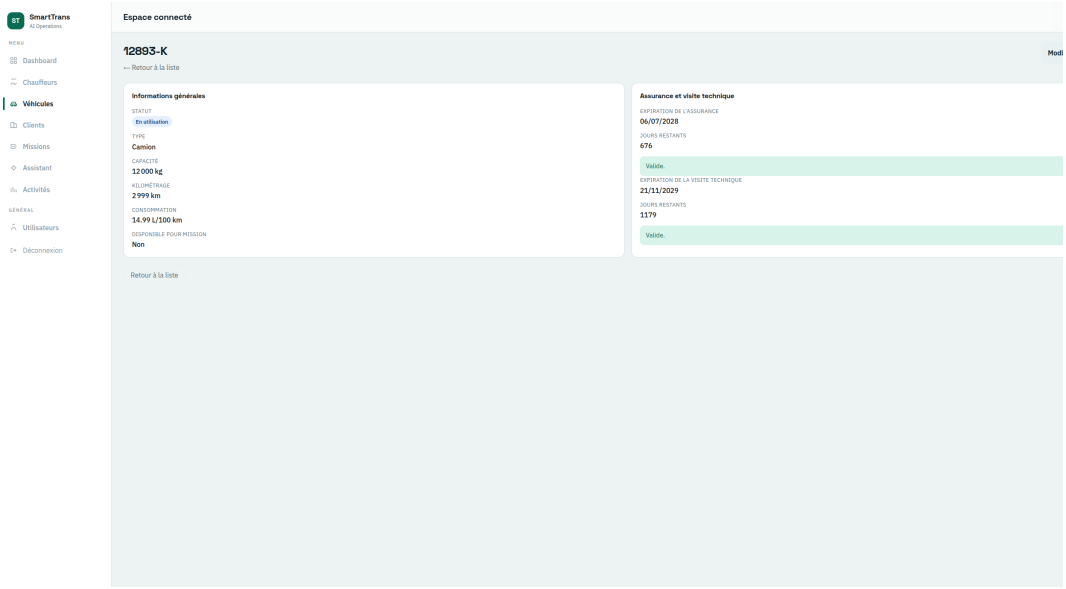


Figure : 7.9 : Fiche détail d'un véhicule

correspondent à ce référentiel, obligatoire à la planification d'une mission.

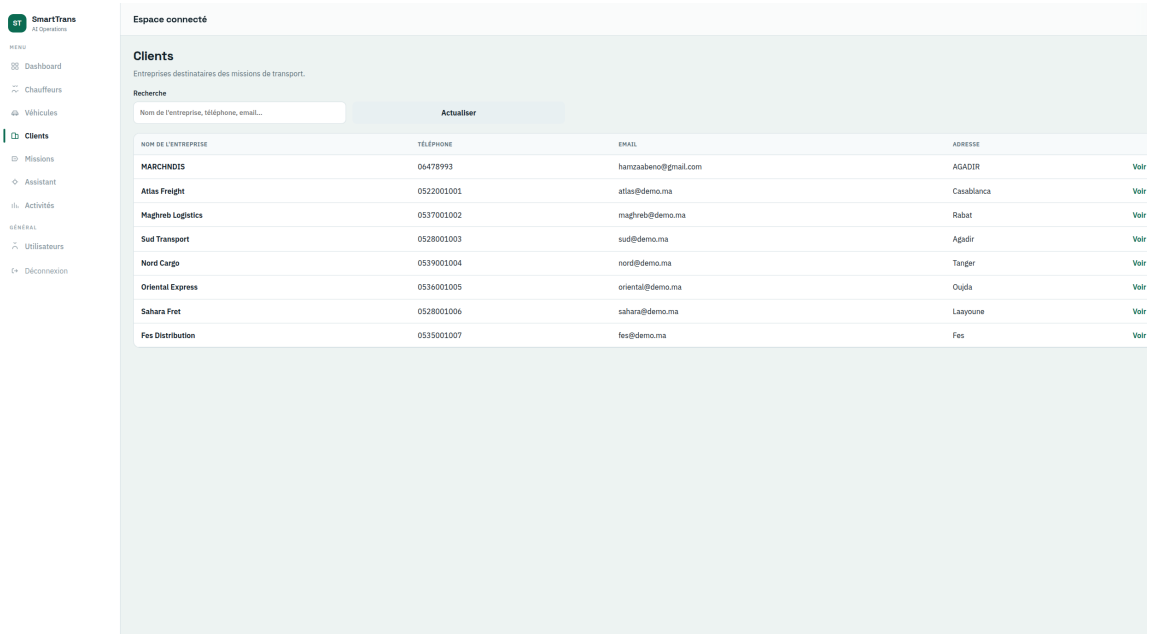


Figure : 7.10 : Liste des clients

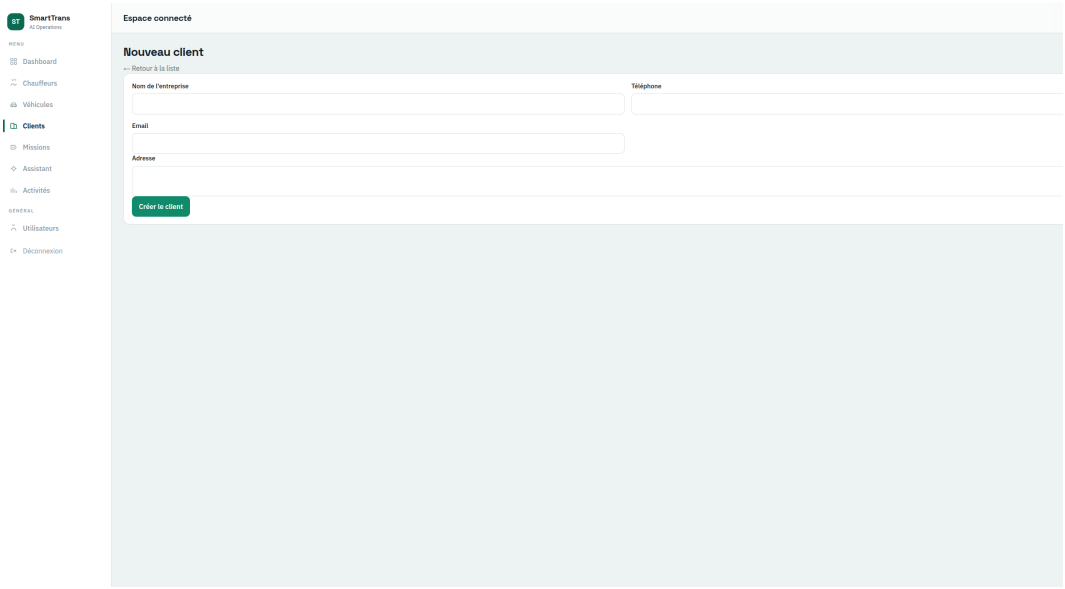


Figure : 7.11 : Formulaire de création d'un client

VII. Planification et suivi des missions

La liste (figure 7.13) relie référence, client, trajet, date, chauffeur, véhicule et statut. Le formulaire de création (figure 7.14) impose une référence, un client et un trajet ; chauffeur et véhicule peuvent rester non assignés. Le détail (figure 7.15) montre l'état d'avancement et les dates de démarrage et de clôture.

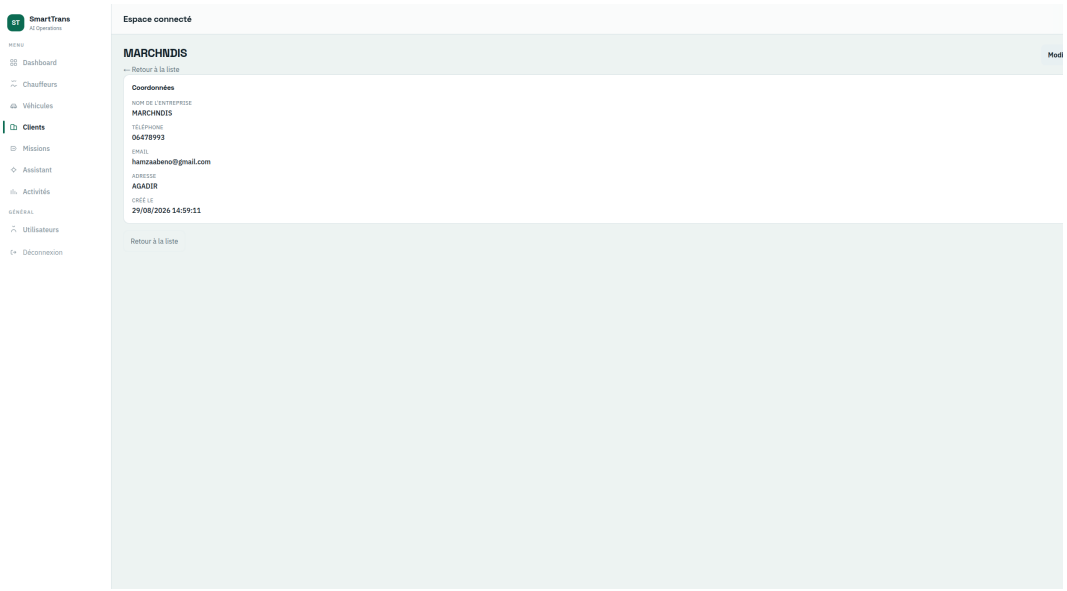


Figure : 7.12 : Fiche détail d'un client

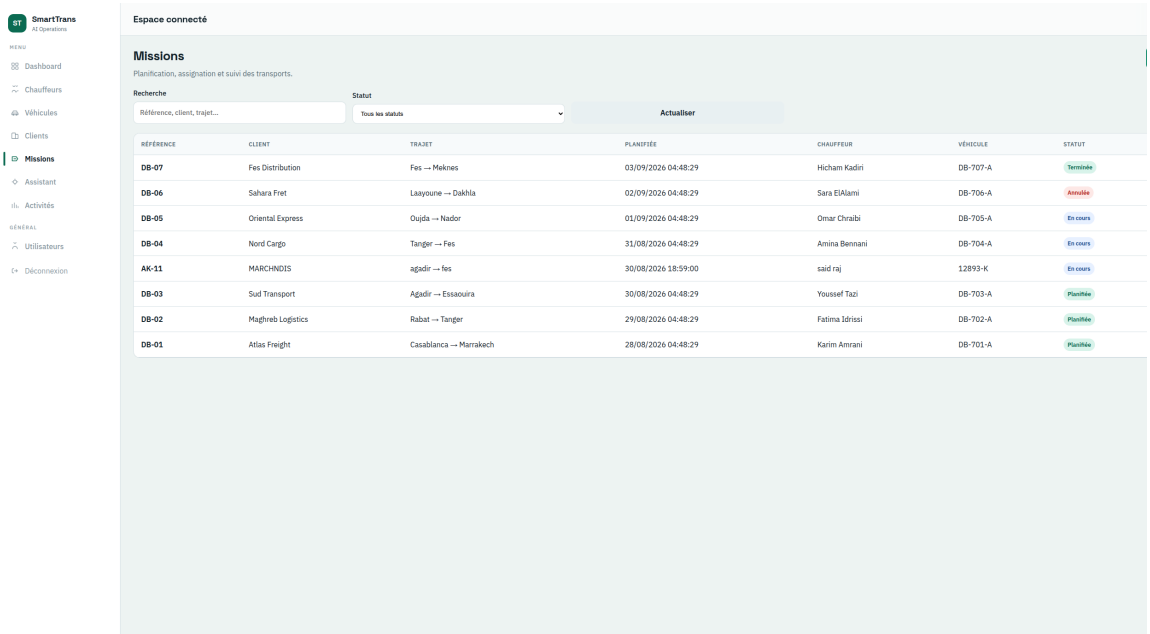


Figure : 7.13 : Liste des missions

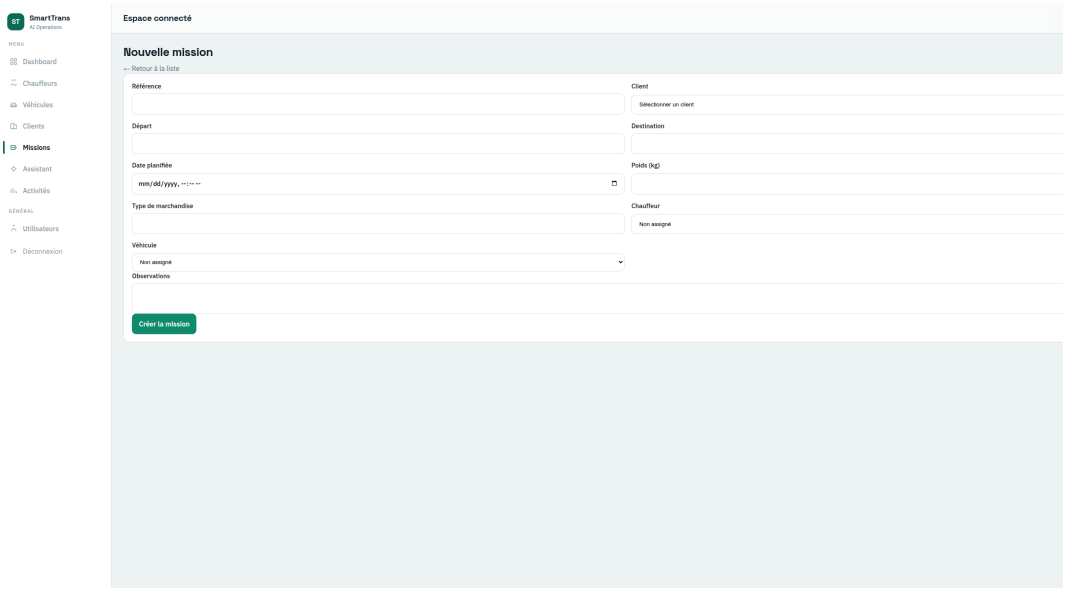


Figure : 7.14 : Formulaire de création d'une mission

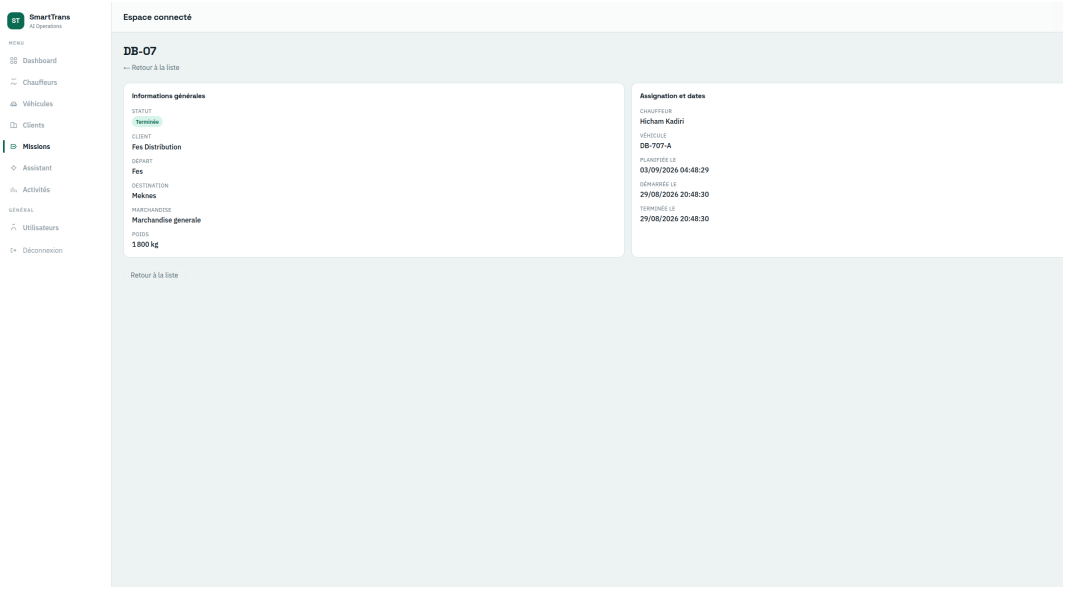


Figure : 7.15 : Fiche détail d'une mission

VIII. Assistant

La figure 7.16 montre l'assistant réservé à l'exploitation. L'utilisateur pose une question en langue naturelle ou choisit une suggestion. La réponse s'appuie sur les données de la base (missions, chauffeurs, véhicules, alertes), afin de limiter les inventions.

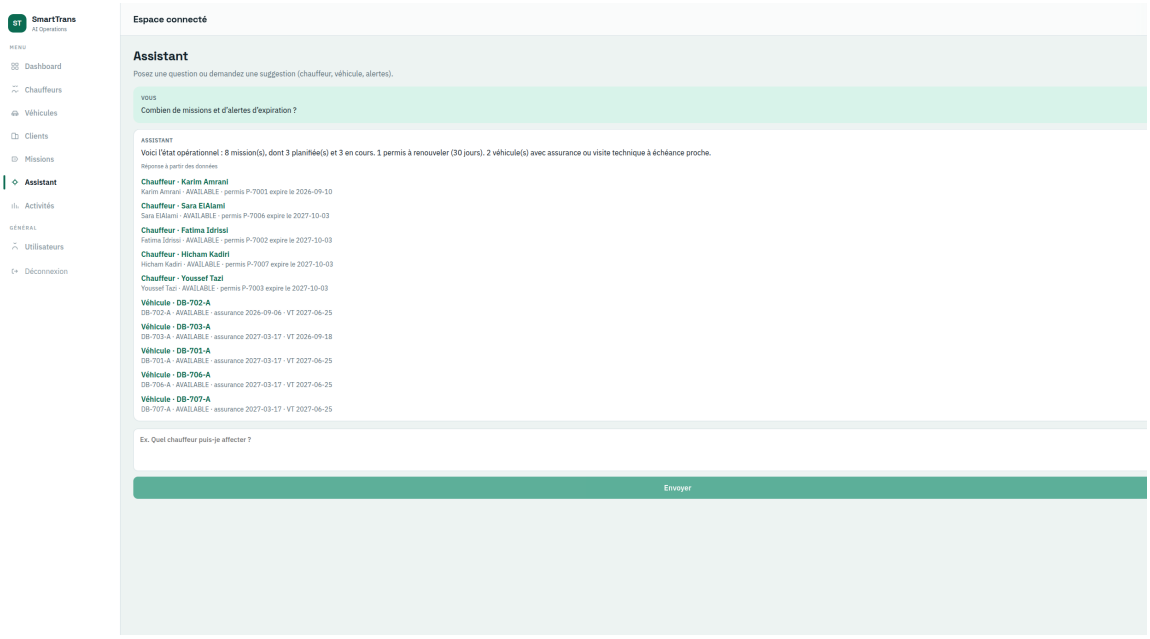


Figure : 7.16 : Assistant conversationnel ancré sur les données métier

IX. Journal d'activités

La figure 7.17 présente le journal en lecture seule. Chaque ligne indique la date, l'action, le type d'entité et un libellé d'objet (courriel, référence de mission, nom, immatriculation). Le filtre par type d'action facilite l'audit.

X. Administration : utilisateurs et rôles

La liste des comptes (figure 7.18) et le formulaire de création (figure 7.19) sont réservés à l'administrateur. Les rôles (figure 7.20) correspondent aux trois profils du système : administrateur, responsable transport, chauffeur.

XI. Conclusion

Les captures confirment la cohérence de l'interface avec l'analyse : navigation par rôle, formulaires alignés sur le modèle de données, tableau de bord orienté alertes, assistant et journal pour le pilotage. La conclusion générale dresse le bilan du stage.

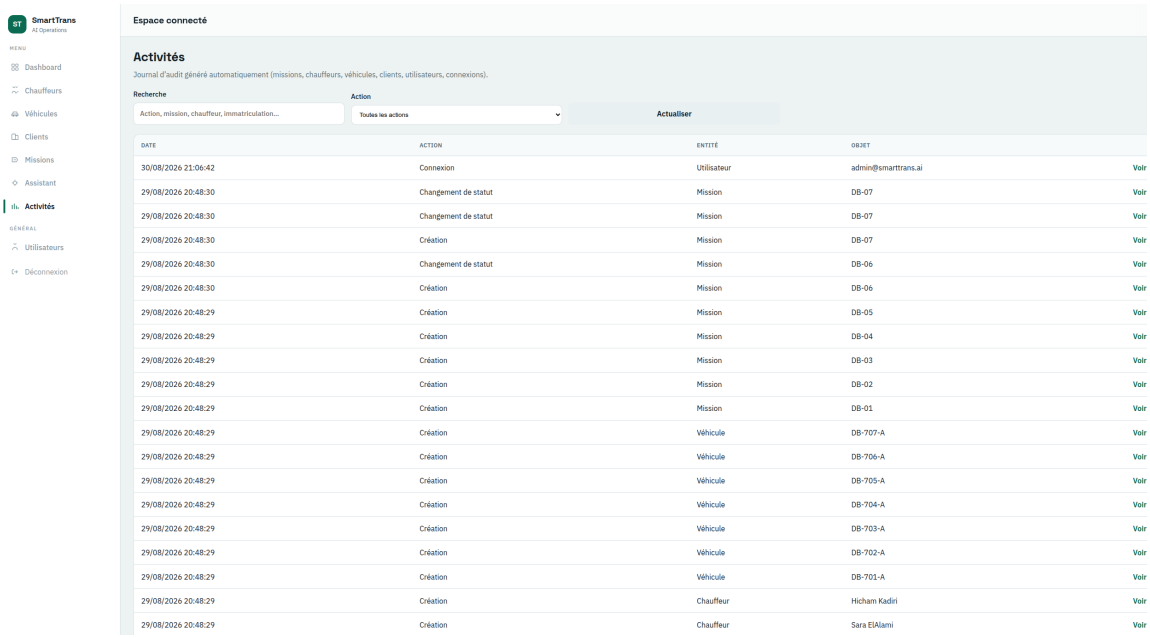


Figure : 7.17 : Journal d'activités (audit automatique)

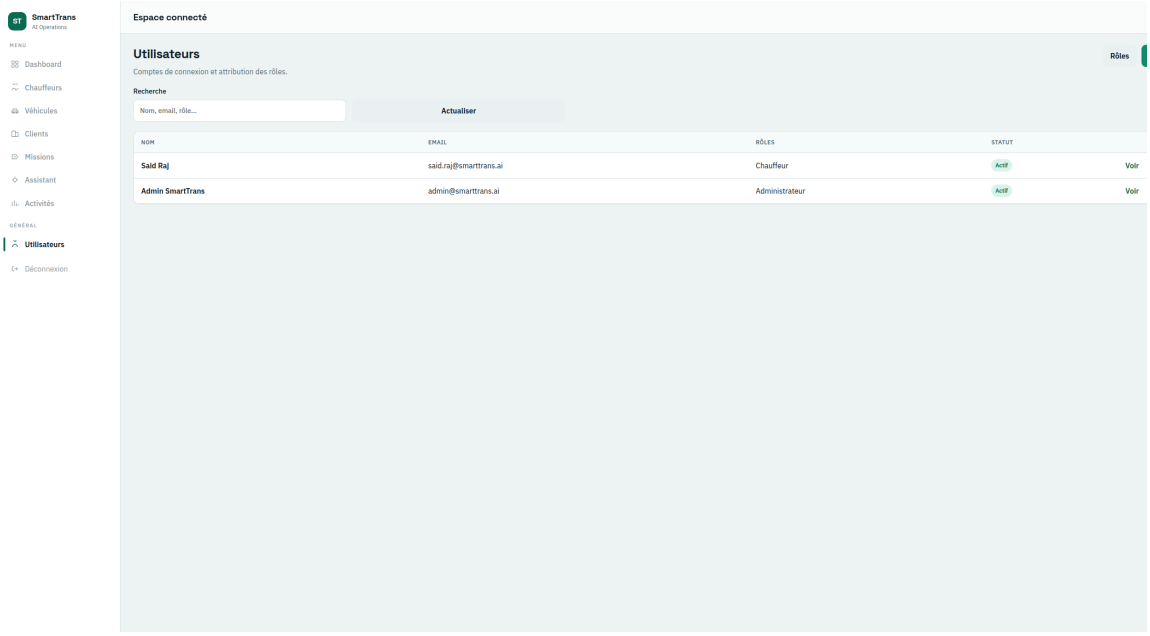


Figure : 7.18 : Liste des utilisateurs

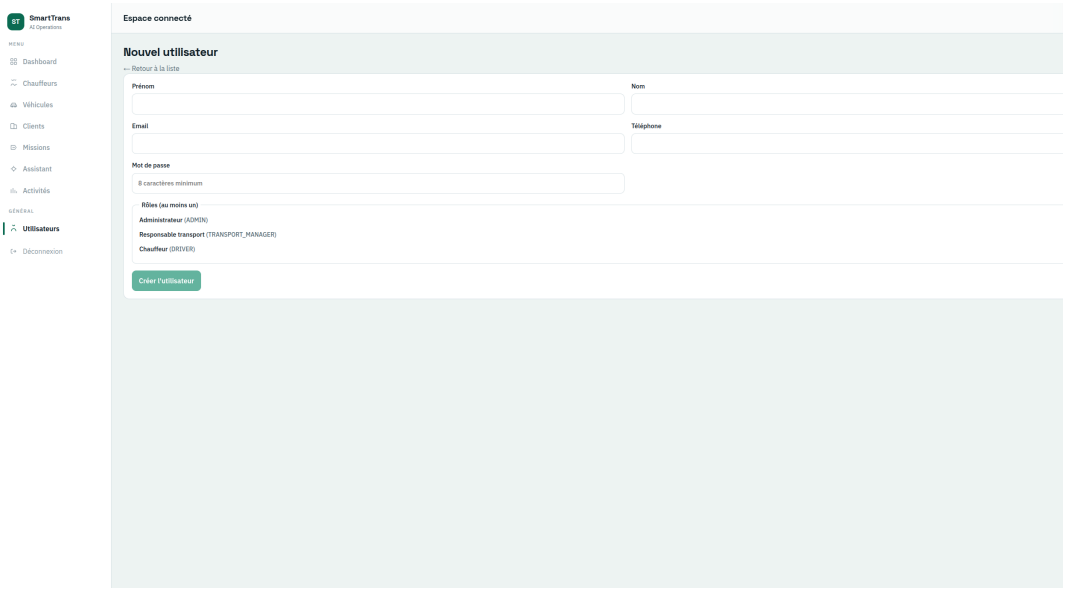


Figure : 7.19 : Formulaire de création d'un utilisateur

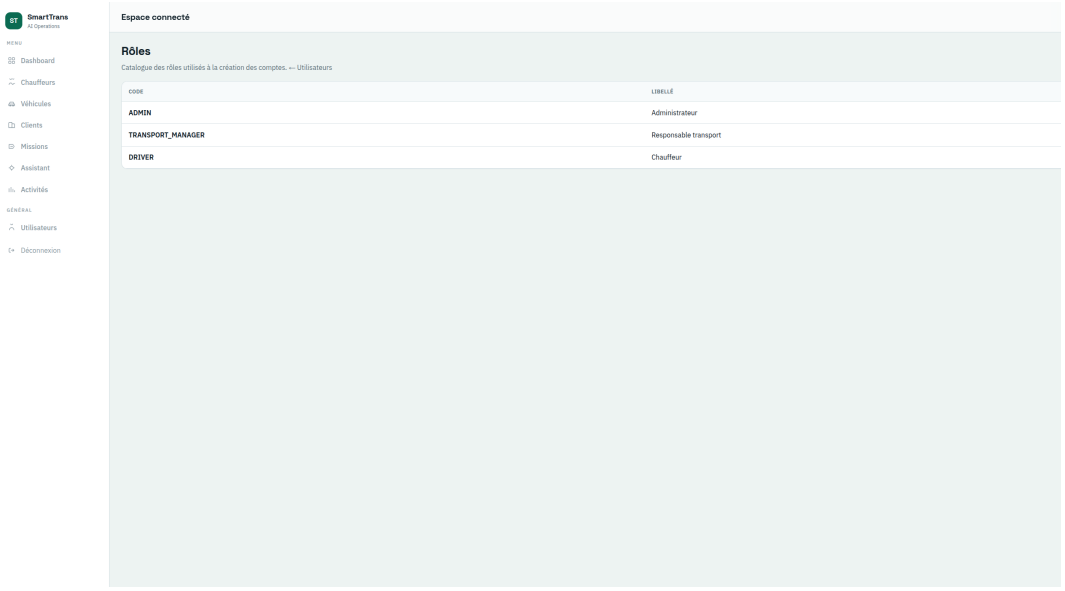


Figure : 7.20 : Catalogue des rôles

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Le stage a permis de concevoir et d'implémenter **SmartTrans-AI**, une application de gestion de transport centrée sur la mission, les rôles et un assistant ancré sur les données. Du point de vue pédagogique, le travail a mobilisé l'état de l'art, la spécification UML, la conception d'une API REST sécurisée, une interface selon le profil utilisateur, une première intégration d'IA contrainte par le SI, ainsi que la conduite d'un projet itératif.

Du point de vue de l'organisme, le livrable offre une base opérationnelle pour le département informatique de S. WATRANS : planification, suivi, alertes d'expiration et traçabilité. Les perspectives immédiates concernent le renforcement des tests métier, le déploiement sécurisé, et d'éventuels modules (notifications, pièces jointes, interfaçage temps réel) si le besoin s'en confirme.

Ce rapport, tenu dans une fourchette d'une quarantaine de pages (limite ENSIASD : soixante-dix pages), constitue la trace écrite de cette réalisation.

BIBLIOGRAPHIE

[1] Spring, Spring Boot Reference Documentation, `https://docs.spring.io/spring-boot/` . 29-08-2026.

[2] Meta Open Source, React Documentation, `https://react.dev/` . 29-08-2026.

[3] PostgreSQL Global Development Group, PostgreSQL Documentation, `https://www.postgresql.org/docs/` . 29-08-2026.

[4] Object Management Group, OMG Unified Modeling Language (OMG UML), Object Management Group, 2017.

[5] JONES M., BRADLEY J., SAKIMURA N., JSON Web Token (JWT), RFC 7519, IETF, 2015.

ANNEXE A

Cette annexe décrit le guide technique de déploiement et d'exécution de SmartTrans-AI.

A.1 Backend

Prérequis

- Java 25 ;
- Maven Wrapper fourni dans le dépôt ;
- PostgreSQL, via Docker Compose en environnement de développement ou via une instance locale.

Lancement

Depuis le répertoire `backend` :

```
./mvnw spring-boot:run
```

Vérification de l'état de l'application

L'état de santé de l'API peut être vérifié à l'adresse :
`http://127.0.0.1:8080/actuator/health`

Documentation de l'API

La documentation OpenAPI/Swagger est accessible à :
`http://127.0.0.1:8080/swagger-ui.html`
L'API est accessible sous le préfixe `/api/v1`.

Compte administrateur de démonstration

Un compte administrateur est disponible en environnement de développement :
`admin@smarttrans.ai`
Le mot de passe est défini dans la configuration locale de l'application.

A.2 Frontend

Depuis le répertoire `frontend`, installer les dépendances :

```
npm install
```

Puis lancer l'application :

```
npm run dev
```

L'interface est accessible à l'adresse :
`http://127.0.0.1:5173`
Le proxy Vite relaie les requêtes `/api` vers le backend exécuté sur le port `8080`.

A.3 Stack Docker

Le fichier `docker-compose.prod.yml` permet d'assembler les principaux services de l'application :

- PostgreSQL ;
- l'API backend ;
- l'interface frontend.

Les informations sensibles, notamment les mots de passe et les clés secrètes, doivent être configurées à l'aide d'un fichier d'environnement non versionné.

ANNEXE B

Cette annexe documente l'API REST de SmartTrans-AI (préfixe `/api/v1`). La description interactive est également disponible via OpenAPI/Swagger : `http://127.0.0.1:8080/swagger-ui.html`.

B.1 Conventions

- Format d'échange : JSON (`Content-Type: application/json`).
- Identifiants : UUID.
- Authentification : en-tête `Authorization: Bearer <jeton>`, sauf `POST /api/v1/auth/login`.
- Rôles : `ADMIN`, `TRANSPORT_MANAGER`, `DRIVER`.

B.2 Authentification

Tableau : 2.1 : Authentification

Ressource	Méthode	Description
`/api/v1/auth/login`	POST	Connexion ; retourne le JWT et le profil

B.3 Utilisateurs et rôles

Les ressources suivantes sont réservées à l'administrateur.

Tableau : 2.2 : Utilisateurs et rôles

Ressource	Méthode	Description
`/api/v1/users`	GET, POST	Liste et création des comptes
`/api/v1/users/{id}`	GET, PATCH, DELETE	Lecture, mise à jour, suppression
`/api/v1/roles`	GET, POST	Liste et création des rôles
`/api/v1/roles/{id}`	GET, PATCH, DELETE	Lecture, mise à jour, suppression

B.4 Référentiels métier

Clients, chauffeurs et véhicules sont gérés par l'administrateur et le responsable transport. Le chauffeur accède à son profil (`/drivers/me`), à la consultation limitée des fiches et à la mise à jour de sa disponibilité.

Tableau : 2.3 : Clients, chauffeurs et véhicules

Ressource	Méthode	Description
`/api/v1/clients`	GET, POST	Liste et création
`/api/v1/clients/{id}`	GET, PATCH, DELETE	Fiche client
`/api/v1/drivers`	GET, POST	Liste et création
`/api/v1/drivers/me`	GET	Profil du chauffeur connecté
`/api/v1/drivers/expiring-licenses`	GET	Permis proches de l'échéance
`/api/v1/drivers/{id}`	GET, PATCH, DELETE	Fiche chauffeur
`/api/v1/drivers/{id}/availability`	PATCH	Disponibilité
`/api/v1/drivers/{id}/deactivate`	PATCH	Désactivation
`/api/v1/vehicles`	GET, POST	Liste et création
`/api/v1/vehicles/expiring-documents`	GET	Assurance / visite technique
`/api/v1/vehicles/{id}`	GET, PATCH, DELETE	Fiche véhicule
`/api/v1/vehicles/{id}/deactivate`	PATCH	Désactivation

B.5 Missions

Le chauffeur peut consulter les missions qui le concernent et enchaîner les statuts autorisés (planifiée → en cours → terminée). L'annulation reste du ressort de l'exploitation.

Tableau : 2.4 : Missions

Ressource	Méthode	Description
`/api/v1/missions`	GET, POST	Liste et planification
`/api/v1/missions/{id}`	GET, PATCH, DELETE	Fiche mission
`/api/v1/missions/{id}/status`	PATCH	Changement de statut

B.6 Journal et assistant

Le journal est en lecture seule. L'assistant est réservé à l'administrateur et au responsable transport.

Tableau : 2.5 : Journal et assistant

Ressource	Méthode	Description
`/api/v1/activities`	GET	Journal d'audit
`/api/v1/activities/{id}`	GET	Détail d'un événement
`/api/v1/assistant/ask`	POST	Question métier ancrée sur les données

B.7 Accès à la documentation interactive

Avec le backend démarré (`./mvnw spring-boot:run`), ouvrir :

`http://127.0.0.1:8080/swagger-ui.html`

L'état de l'API est indiqué par `http://127.0.0.1:8080/actuator/health`.